

University of Colombo School of Computing

SCS1301 - Data Structures and Program Design using C

Y1 S1 Assignment report (Ludo)

**Done by: AMKS ATTANAYAKE
Student ID :- 23000139**

Terms of Reference

A report submitted in fulfilment of the requirement for the module
SCS 1301 - Data Structures and Program Design using C, University
of Colombo School of Computing

Table of Contents

Planning process	4
1.Classical Ludo Flow Chart.....	5
2.Player behaviors flow charts.....	6
2.1 Red Player	6
2.2 Yellow Player	7
2.3 Green Player	8
Thoughts on the Project's Initial Flowcharts and Their Effect	9
Code Justification.....	10
Usage of Structures.	18
Identified Inefficiencies and challanges.....	18
Appendices.....	19

Planning process

I tried to use problem-solving approaches like decomposition, divide and conquer, pattern recognition, and other methods when I approached this task.

Even yet, it was difficult to approach the issue because there were many questions, such where to begin and what has to be there. It would be better to use an array, etc.

I therefore made the decision to use the traditional approach and begin by simply creating very basic flow charts in order to begin solving the chores.

As a result, I have broken down the issue into numerous flow charts. These diagrams of flow are,

The following flow charts are available:

- Red player behavior,
- Yellow player behavior
- Green player behavior and
- Classical ludo.

These flow charts are really basic and were created purely to help me understand how to approach the subject, as I previously mentioned. It represents what should be there not how should be.

1. Classical Ludo Flow Chart

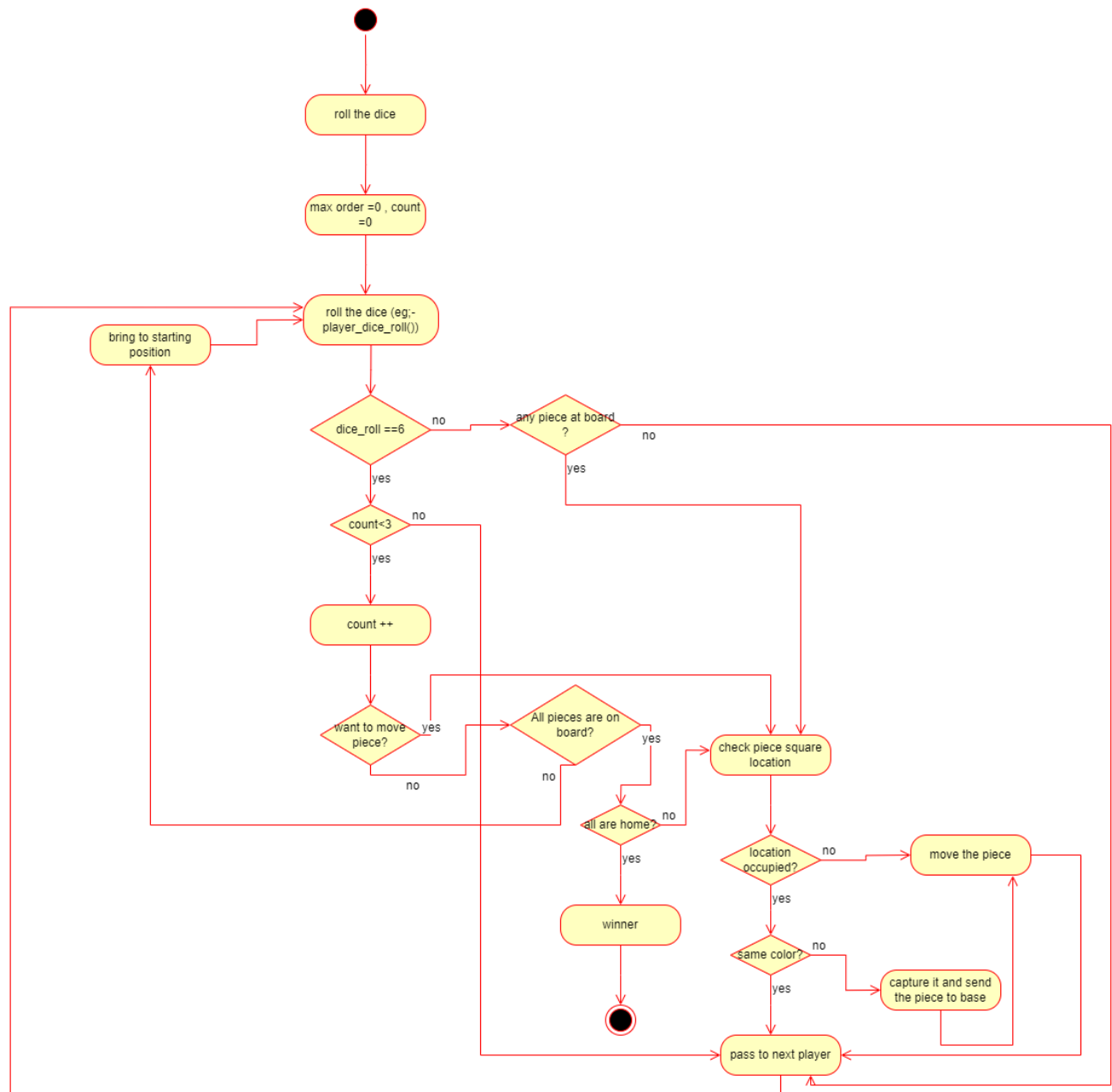


Figure 1 . classical ludo flow chart

This flow chart, as I see, captures main functions that needed to be in the classical ludo game.

2.Player behaviors flow charts

2.1 Red Player

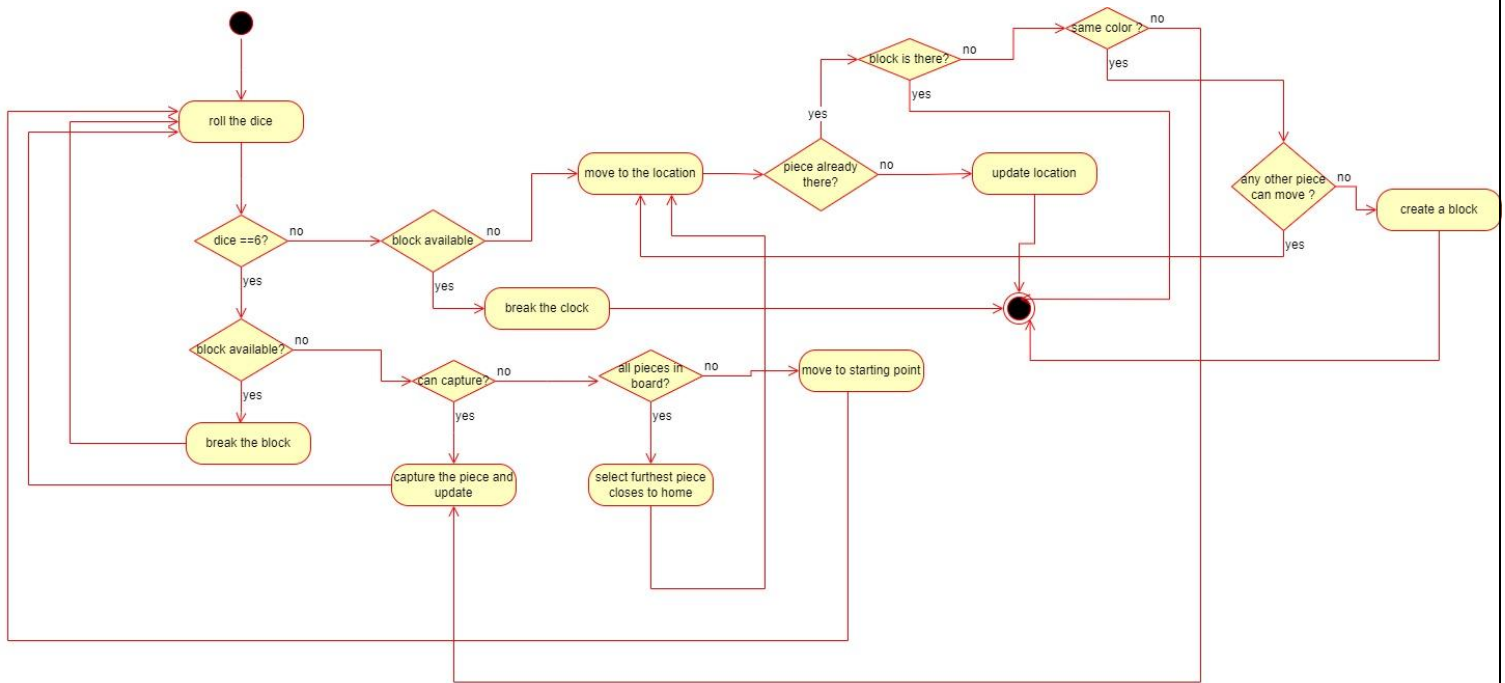


Figure 2 (red player flow chart)

This flow chart captures the aggressive playing nature of the red player. The red player's strategy will be to win the game by playing aggressively by capturing when has a chance and creating a block when there are no possible moves to make.

2.2 Yellow Player

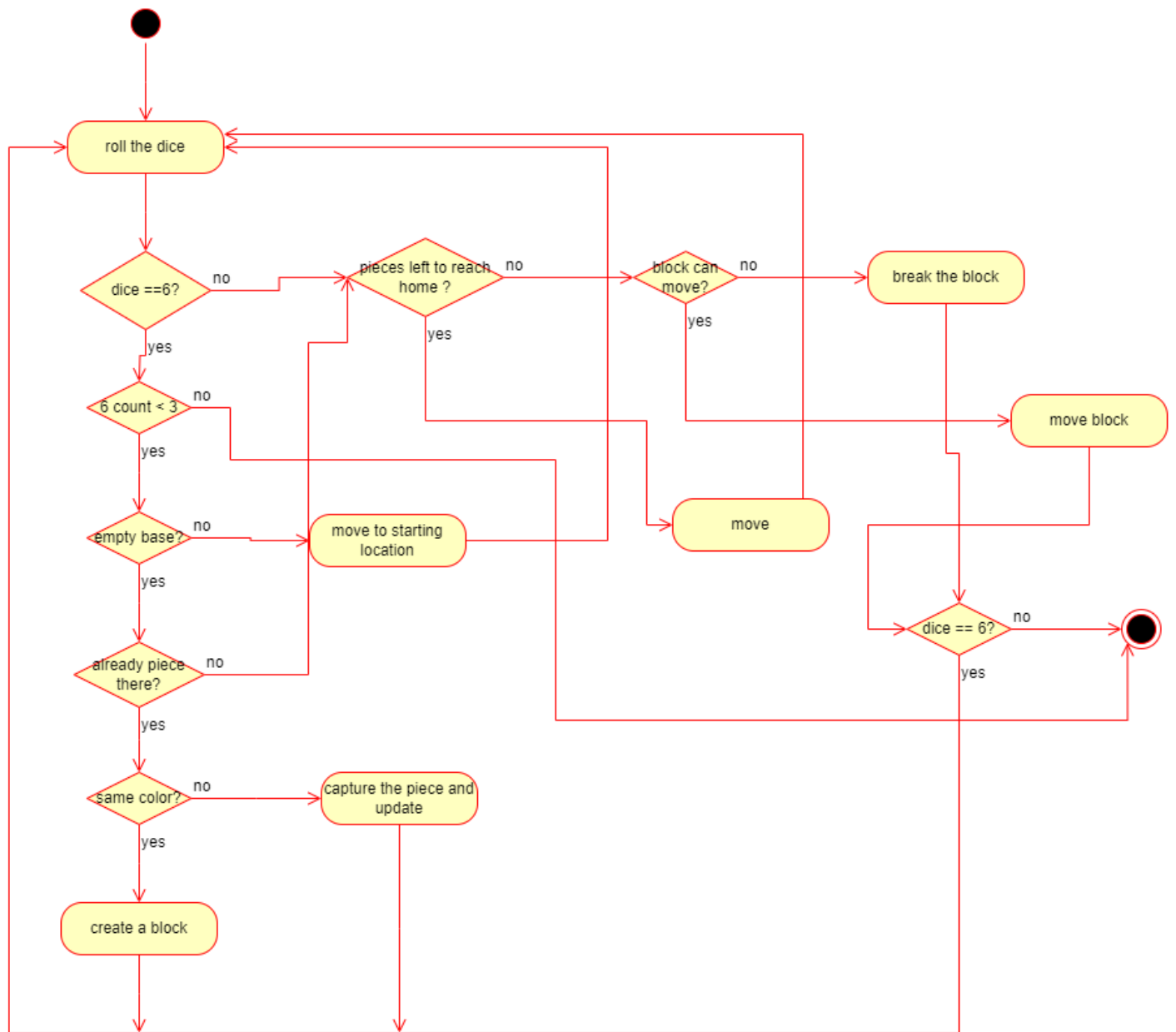


Figure 3 (yellow player flow chart)

This flow chart shows the yellow players' playing behavior. Always trying to empty the base and capturing processes and creating blocks are shown.

2.3 Green Player

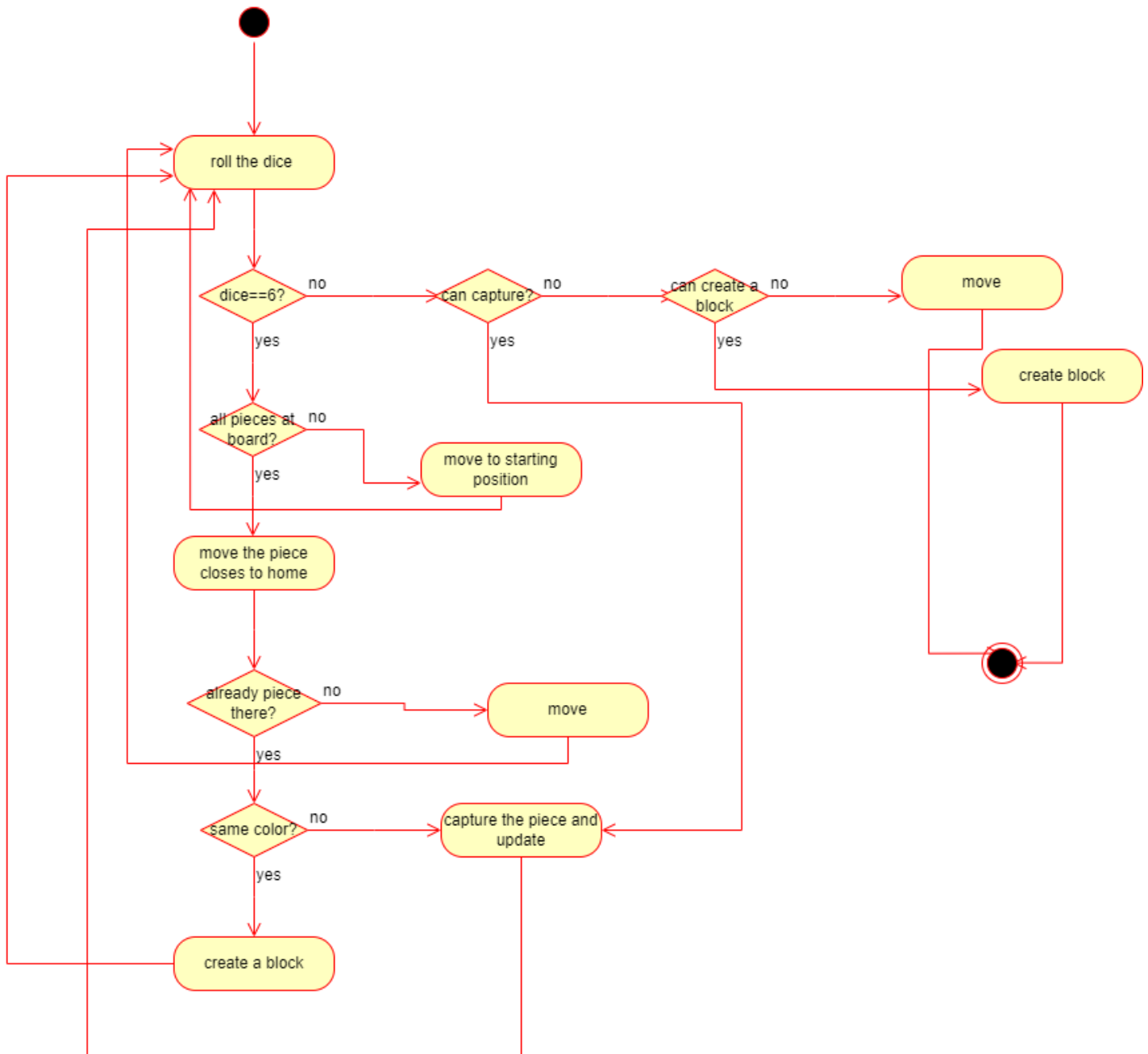


Figure 4 (green player flow chart)

This shows the prioritizing moving piece to the home behavior and creating block behavior of green.

Thoughts on the Project's Initial Flowcharts and Their Effect

A fundamental, if somewhat simplistic, depiction of the project's procedures was given by the flowcharts that were previously supplied (Figures 1 to 4). The development process benefited greatly from these diagrams, even though they might not accurately depict the subtleties and complexity of the finished product.

Even though they were straightforward, these early flowcharts helped direct my design and coding efforts in the right directions. They made it possible for me to see how the software is organized and flows generally, which made it easier for me to pinpoint its main elements and possible problems. Flowcharts offered a road map for the creation of more intricate and precise algorithms by decomposing the problem into smaller, more manageable components.

The flowcharts have some advantages despite their shortcomings in terms of precision and detail:

Conceptual Clarity: They supported the establishment of a clear foundation for subsequent development and ideation by elucidating the basic ideas and procedures that had to be covered in the code.

The identification of core processes was facilitated by the diagrams, which brought together important activities including player movement, dice rolling mechanics, and piece status management. These processes later became central to the development pipeline.

Iterated Improvement: An iterative approach to problem-solving was encouraged by the flowcharts' preliminary character. More accurate and effective solutions resulted from revisiting and refining these designs as the project moved on.

In summary, even if the original flowcharts may not have included every aspect of the finished product, they played a crucial role in setting the my approach of project's foundation. With their guidance and attention, I was able to approach the problem methodically, try out several alternatives, and create a reliable and useful application. The value of visual planning tools in the software development process is shown by the insights these diagrams provided, which were crucial in turning abstract concepts into usable code.

Code Justification

1. Player Status Loop:

- The function begins by looping through all the players using a for loop (for(int i=0; i<NUM_PLAYERS; i++)).
- For each player, it prints out the number of pieces on the board and in the base using the player's color, piecesInBoard, and piecesInBase attributes.

2. Piece Location Information:

- After displaying the general status of the player's pieces, it prints a header indicating that the locations of the player's pieces will be listed.
- The function then loops through each of the player's pieces (for(int j=0; j<4; j++)), checking the status of each piece and printing its location or position accordingly:
 - **Base:** If the piece is in the base (status == BASE), it prints that the piece is in the base.
 - **Home:** If the piece has reached home (status == HOME), it prints that the piece is in home.
 - **Home Straight:** If the piece is in the home straight (status == HOME_STRAIGHT), it prints that the piece is in the home straight.
 - **Board:** If the piece is on the board (status == BOARD), it prints the piece's location on the board.

3. findPlayerIndexByColor(Player players[], const char* color)

- **Purpose:** Finds the index of a player based on their color.
- **Functionality:**
 - Iterates through the players[] array.
 - Compares each player's color with the provided color string using strcmp().
 - If a match is found, returns the index of that player.
 - If no match is found, returns -1 to indicate the color was not found.

4. moveToStartingpoint(Player *player, int pieceIndex) (this is an underworking function)

- **Purpose:** Moves a player's piece from the base to the board's starting position.
- **Functionality:**
 - Sets the direction of the piece using the toss() function (determines whether it will move clockwise or counterclockwise).

- Updates the piece's status to BOARD, indicating it is now on the board.
- Adjusts the player's count of pieces in the base and on the board.
- Sets the piece's location based on the player's color (e.g., RX for Red, YX for Yellow).
- Increments the xpass attribute if the piece will move clockwise.
- Prints a message showing the piece's new position and movement direction.

5. initializePlayer(Player* player, const char* color)

- **Purpose:** Initializes a player's attributes and their pieces at the start of the game.
- **Functionality:**
 - Copies the player's color string to the player's color attribute.
 - Sets the player's homeStart location based on their color.
 - Initializes each piece:
 - Assigns a unique ID (e.g., "R1" for Red piece 1).
 - Sets the piece's initial location to BASE_LOCATION.
 - Sets the piece's status to BASE, indicating it starts in the base.
 - Initializes attributes like captured, direction, xpass, and various status flags (isBlocked, isEnergized, isSick, skipRounds).
 - Initializes counters like piecesInHome, piecesInBoard, and piecesInBase.

6. move(Player* player, Player players[], int numPlayers)

- **Purpose:** Directs the player's move to the correct color-specific function.
- **Functionality:**
 - Checks the player's color using strcmp().
 - Calls the corresponding move function (moveRed, moveYellow, moveGreen, or moveBlue) based on the player's color.

7. rollDice()

- **Purpose:** Simulates rolling a six-sided dice.
- **Functionality:**
 - Generates a random number between 1 and 6 using rand() % 6 + 1.
 - Returns the result of the dice roll.

8. capturePiece(Player* player, Player players[], int numPlayers)

- **Purpose:** Handles the capture of an opponent's piece when a player's piece lands on the same square.
- **Functionality:**
 - Loops through all opponents (other players).
 - For each opponent, checks if any of their pieces are on the board (status == BOARD) and if their location matches the current player's piece location.
 - If a match is found, it indicates a capture:
 - The opponent's piece is returned to the base (status = BASE and location = BASE_LOCATION).
 - Updates the opponent's counters (piecesInBoard decreases, piecesInBase increases).
 - Prints a message showing which piece was captured and its new location.

9. handleMysteryCell(Player *player, int pieceIndex, int mysteryEffect)

- **Purpose:** Applies the effect of landing on a mystery cell.
- **Functionality:**
 - Based on the mysteryEffect value, the function applies one of several possible effects:
 1. **Teleport:** Moves the piece forward by 10 positions.
 2. **Energized:** Doubles the piece's movement speed (isEnergized = 1).
 3. **Sick:** Halves the piece's movement speed (isSick = 1).
 4. **Skip Rounds:** Prevents the piece from moving for four rounds (skipRounds = 4).
 5. **Return to Base:** Sends the piece back to the base (location = -1, status = BASE, updates counters).
 6. **Change Direction:** Reverses the piece's direction (direction *= -1).
 - Prints a message describing the effect applied.

10. calculateDistancetoHome(Player* opponent, int pieceIndex)

- **Purpose:** Calculates the distance of a piece from its current position to its home.
- **Functionality:**
 - Depending on the piece's direction (clockwise or counterclockwise), calculates the distance to the home start position.
 - For clockwise movement, the distance is calculated as the sum of the remaining board size and the difference between the home start and the current location.
 - For counterclockwise movement, it's the difference between the current location and the home start, plus the board size.
 - Returns the calculated distance.

11. attemptCapture(Player* player, Player players[], int numPlayers, int dice)

- **Purpose:** Tries to capture an opponent's piece by moving the player's piece after rolling the dice.
- **Functionality:**
 - Iterates over each of the player's pieces to determine if a capture is possible.
 - For each piece on the board:
 - Calculates the new location after moving by the number rolled on the dice (new_location).
 - Checks if any opponent's piece is already at that new location.
 - If a capture is possible, it compares the distance to home (using calculateDistancetoHome) and identifies the best piece to move.
 - If a capture is possible (bestPieceIndex != -1):
 - Moves the player's piece to the new location.
 - Calls capturePiece() to handle the capture.
 - Prints a message indicating the capture and returns 1 to indicate success.
 - If no capture is possible, returns 0.

12.moveRed(Player* player, Player players[], int numPlayers)

This function handles the movement of the red player's pieces. Here's how it works:

1. Rolling the Dice:

- The dice is rolled using the rollDice() function, and the result is stored in dice.
- If the dice roll is 6, the roll6count is incremented. If the player rolls three consecutive 6s, their turn ends immediately, and the function returns.

2. Piece Movement Logic:

- The function checks each piece to determine its current status:
 - **Base to Board:** If the piece is in the base and the dice roll is 6, the piece is moved to the board. Its location is set to RX (starting position for the red player), and its direction is determined by a coin toss (toss() function). The direction can be either clockwise or anticlockwise.
 - **Board Movement:** If the piece is already on the board, it moves forward according to the dice roll, modified by its direction (clockwise or anticlockwise). If the piece is energized or sick, the dice roll is doubled or halved, respectively.
 - **Home Straight:** If the piece reaches the home straight (i.e., it's near the home position), the function checks if the piece can enter it. The piece's status is updated accordingly.

3.Capturing Logic:

- The function checks if a capture can be made by the red player's piece after moving. The capture logic would involve sending an opponent's piece back to the base if they land on the same square.

4.Ending the Turn:

- The loop continues as long as the player rolls a 6, allowing them to move multiple pieces in one turn. If the roll is anything other than 6, the loop breaks, and the turn ends.

13.moveYellow(Player* player, Player players[], int numPlayers)

This function handles the movement of the yellow player's pieces, following similar logic to moveRed with some variations specific to the yellow player's strategy.

1. Rolling the Dice:

- The dice is rolled, and if the player rolls three consecutive 6s, their turn ends immediately.

2. Piece Movement Logic:

- **Base to Board:** If a piece is in the base and the dice roll is 6, the piece is moved to the board. Its starting location is set to YX (the starting position for the yellow player), and its direction is set by a coin toss.
- **Board Movement:** Pieces already on the board move according to the dice roll. The direction (clockwise or anticlockwise) affects the movement, and energized or sick pieces adjust the dice roll accordingly.
- **Home Straight:** If the piece can move to the home straight, it does so, and its status is updated.

3. Prioritizing Pieces:

- Unlike the red player, the yellow player may prioritize moving the piece closest to the home.

4. Ending the Turn:

- The yellow player's turn ends when a non-6 is rolled, similar to the red player's logic.

14.moveGreen(Player* player, Player players[], int numPlayers)

This function controls the movement of the green player's pieces. Here's how it works:

1. Rolling the Dice:

- The function begins by rolling the dice with the rollDice() function. If the roll is a 6, the player gets to roll again. However, if the player rolls three consecutive 6s, their turn ends immediately.

2. Piece Movement Logic:

- The function then iterates over each piece belonging to the green player to determine its current status and whether it can be moved:
 - **Base to Board:** If the piece is still in the base and the dice roll is 6, the piece is moved to the board at the green player's starting position (GX). The piece's direction (clockwise or anticlockwise) is decided by a coin toss (toss() function). If the direction is clockwise, the xpass (a counter for passing the starting position) is incremented. The function also updates the number of pieces on the board and in the base.
 - **Board Movement:** If the piece is already on the board, it moves forward based on the dice roll, adjusted by its direction and any status effects like being energized (doubles the dice roll) or sick (halves the dice roll). The new location of the piece is calculated, and if necessary, the piece's location is adjusted for moving in the opposite direction.
 - **Home Straight:** If the piece is near the home start and conditions are met (e.g., it has passed the home start at least once), it can move to the home straight. The function checks whether the piece can enter the home straight

based on its direction and updates the piece's status and location accordingly.

3. **Capturing Logic:**

- After moving a piece, the function checks if it can capture an opponent's piece by landing on the same square. If a capture is made, the opponent's piece is sent back to the base.

4. **Home Movement:**

- If the piece is on the home straight, it moves according to the dice roll. The piece must land exactly on the final home position to enter home. If the roll would overshoot the home position, the move is undone.

5. **Ending the Turn:**

- The turn continues if the player rolls a 6; otherwise, the turn ends.

15.moveBlue(Player* player, Player players[], int numPlayers)

This function controls the movement of the blue player's pieces, following a similar logic to moveGreen but with adjustments for the blue player's starting position and strategy.

1. **Rolling the Dice:**

- The dice is rolled, and if the player rolls three consecutive 6s, the turn ends.

2. **Piece Movement Logic:**

- **Base to Board:** If the piece is in the base and the dice roll is 6, the piece moves to the board at the blue player's starting position (BX). The direction is set by a coin toss, and if the direction is clockwise, the xpass counter is incremented. The function updates the number of pieces on the board and in the base.
- **Board Movement:** The piece moves according to the dice roll, adjusted by its direction and status effects. The new location is calculated, and adjustments are made for direction changes.
- **Home Straight:** The function checks if the piece can enter the home straight. If the conditions are met, the piece moves to the home straight, and its status and location are updated.

3. **Capturing Logic:**

- The function checks for and handles captures of opponent pieces.

4. **Home Movement:**

- On the home straight, the piece moves according to the dice roll. The piece must land exactly on the final home position to enter home. If the move overshoots, it is undone.

5. **Ending the Turn:**

- The turn continues if the player rolls a 6; otherwise, the turn ends.

16. **Player Status Loop:**

- The function begins by looping through all the players using a for loop (for(int i=0; i<NUM_PLAYERS; i++)).
- For each player, it prints out the number of pieces on the board and in the base using the player's color, piecesInBoard, and piecesInBase attributes.

17. **Piece Location Information:**

- After displaying the general status of the player's pieces, it prints a header indicating that the locations of the player's pieces will be listed.
- The function then loops through each of the player's pieces (for(int j=0; j<4; j++)), checking the status of each piece and printing its location or position accordingly:
 - **Base:** If the piece is in the base (status == BASE), it prints that the piece is in the base.
 - **Home:** If the piece has reached home (status == HOME), it prints that the piece is in home.
 - **Home Straight:** If the piece is in the home straight (status == HOME_STRAIGHT), it prints that the piece is in the home straight.
 - **Board:** If the piece is on the board (status == BOARD), it prints the piece's location on the board.

Usage of Structures.

In my code I used 2 structures for player class and pieces class and typedef them to enhance the readability. I preferred use to structure since its clearer and more precise. And often used player structure as a pointer to avoid any unnecessary memory wastage, and sometimes used arrays to clearly identify the player's pieces.

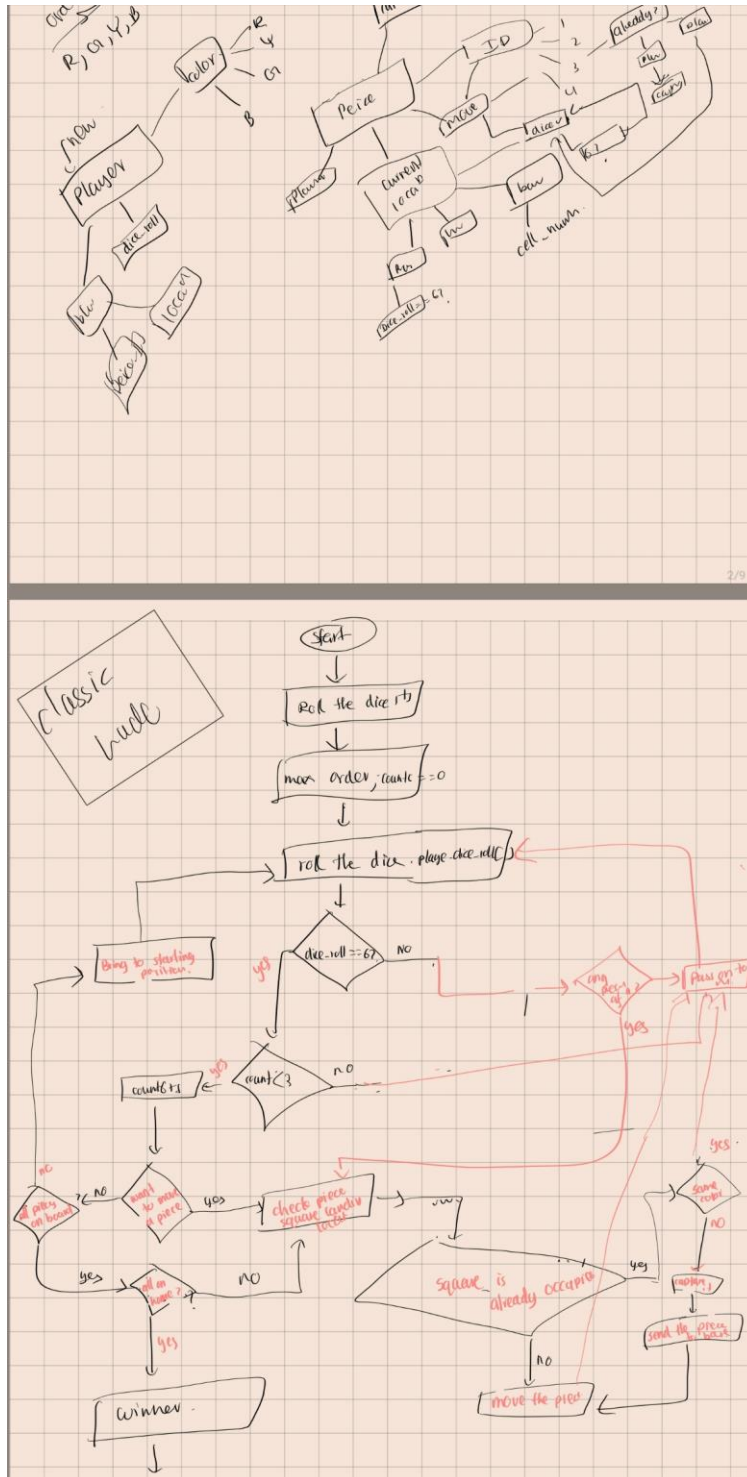
Pieces structure is a property of player as an array of 4 since it has 4 pieces, and each piece has common set of properties. Due to that I preferred use to create a structure rather than creating multi-dimensional array, since if I did, I would mean I need some considerable number of nested loops to access a particular feature of the piece. Which means it would increase the time complexity as well.

Identified Inefficiencies and challenges

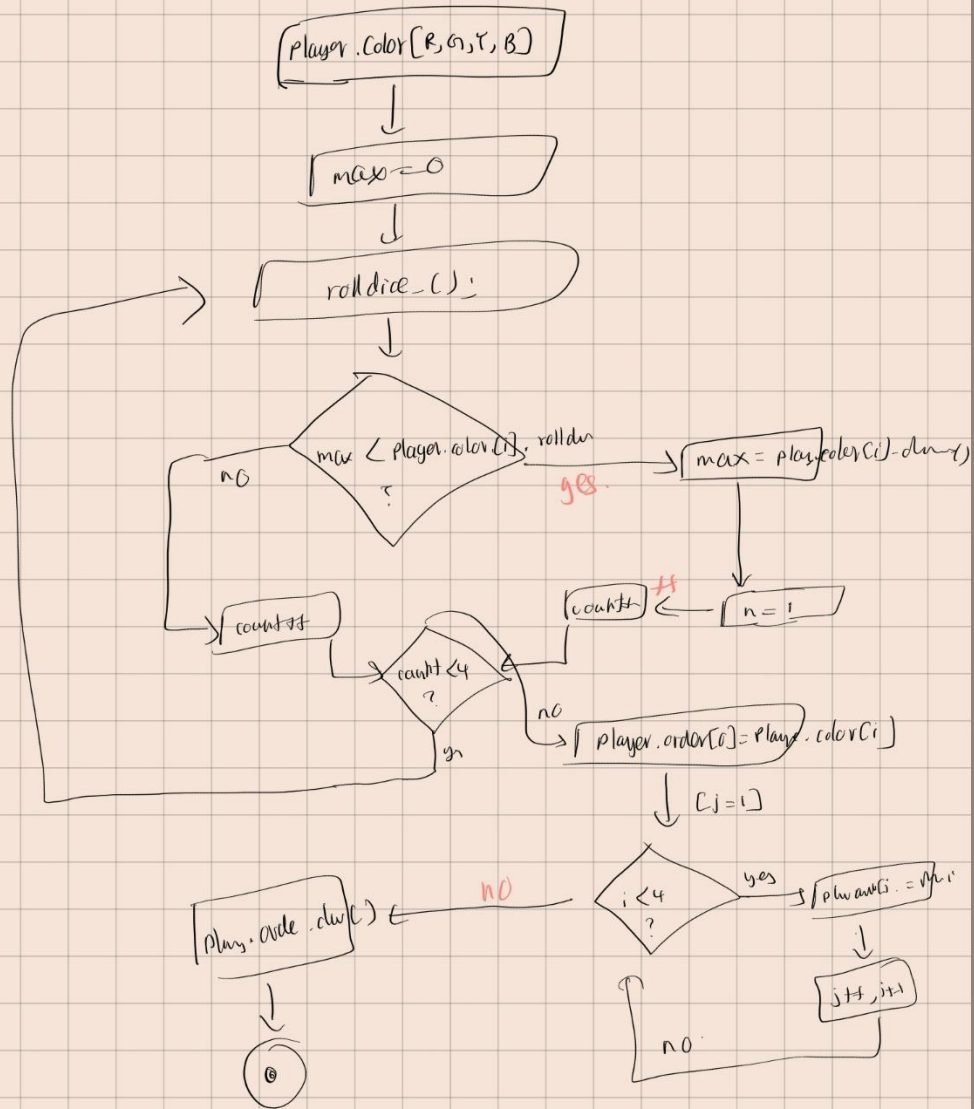
During the development of my code, several inefficiencies and challenges were identified that impacted the overall performance and completeness of the project.

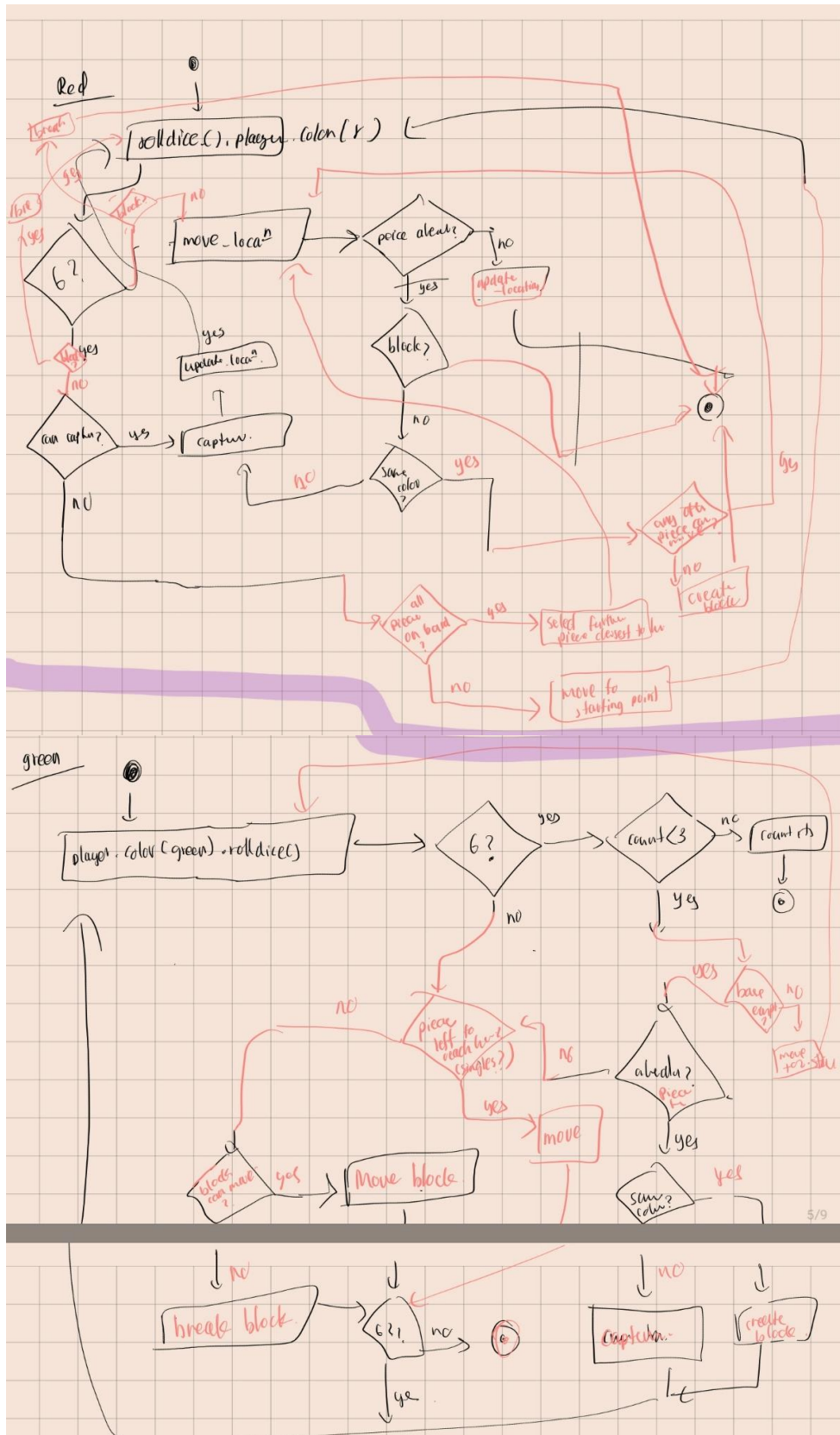
1. **Memory Inefficiency:** The current implementation of the code is not optimized for memory usage. Given more time, I would have explored strategies to reduce memory consumption.
2. **Redundancy:** The codebase contains a significant amount of redundancy, with similar code blocks repeated across different sections. While I attempted to address this issue by creating more generalized functions, time constraints limited the extent of these improvements. The commented-out functions in the code are part of this effort.
3. **Player Logic:** The logic for player movement and interactions is not fully accurate. Specifically, handling of certain game mechanics, such as blocking, remains unimplemented. This is an area that requires further development to achieve a more complete and realistic gameplay experience.
4. **Underdeveloped Functions:** Some functions, which were intended to improve the code structure and efficiency, remain underdeveloped and have been commented out in the code. These functions were not fully realized within the project timeline but are noted for future work.

Hand drawn initial flow charts



roll the dice





yellow

merge.colour(gm).yellow()

G?

all pieces at path?

move to x

move closest to h

can capture?

move closer

already?

same?

how less

block

can m

blue

now?

creat a bh

father

move

